

*Projet Numérique Individuel*

# Équations Différentielles Stochastiques

Analyse théorique et simulation numérique

## Thèmes abordés

Existence et unicité (itération de Picard) · Convergence EDS  $\rightarrow$  EDO  
Dépendance en la condition initiale · Schémas d'Euler-Maruyama et  
Milstein

## Étudiantes

Ndeye Amy DIOP  
Néné KONTE  
Ndèye Sira NDIAYE

## Encadrant

Clément FOUCART

# Introduction

Les équations différentielles stochastiques sont un outil qui permet de modéliser des phénomènes dont l'évolution est soumise à un aléa. Contrairement aux équations différentielles ordinaires, elles intègrent un terme de bruit, typiquement un mouvement brownien, ce qui les rend bien plus adaptées à la description de systèmes réels, qu'il s'agisse de modèles en biologie, en physique ou en finance.

Dans le cadre de ce projet, on a pu étudier ces équations à la fois d'un point de vue théorique et numérique. On commence par rappeler les résultats fondamentaux : existence et unicité de la solution forte sous hypothèse de Lipschitz, démontrés via l'itération de Picard. On s'intéresse ensuite à quelques propriétés importantes des solutions, leur comportement quand le bruit tend vers zéro, et leur sensibilité à la condition initiale. Un exemple concret d'EDS à coefficients non globalement lipschitziens est aussi traité, ce qui oblige à modifier légèrement les coefficients pour pouvoir appliquer le théorème.

La deuxième partie du projet est consacrée à la simulation numérique. On implémente le schéma d'Euler-Maruyama en Python et on discute des difficultés pratiques qui apparaissent, notamment quand le coefficient de diffusion n'est pas défini partout. On compare enfin ce schéma avec celui de Milstein sur un exemple où la solution exacte est connue, ce qui permet de vérifier les ordres de convergence attendus.

## 1 Rappels sur les EDS

Une équation différentielle stochastique, ou EDS, est une équation différentielle dans laquelle on ajoute un terme aléatoire. Elle permet de modéliser l'évolution d'un phénomène soumis à une incertitude ou à un bruit.

Dans ce projet, on considère principalement des EDS de la forme :

$$dX_t = \sigma(X_t) dB_t + b(X_t) dt, \quad X_0 = x.$$

Cette écriture est appelée forme différentielle de l'EDS.

Dans cette équation,  $(B_t)_{t \geq 0}$  est un mouvement brownien. Il représente le bruit aléatoire du modèle.

La fonction  $b$  est appelée coefficient de dérive. Elle décrit la tendance moyenne de l'évolution du processus.

La fonction  $\sigma$  est appelée coefficient de diffusion. Elle mesure l'intensité du bruit aléatoire.

La même EDS peut aussi s'écrire sous forme intégrée :

$$X_t = x + \int_0^t \sigma(X_s) dB_s + \int_0^t b(X_s) ds.$$

Cette forme signifie que la valeur de  $X_t$  est obtenue à partir de la valeur initiale  $x$ , à laquelle on ajoute deux contributions :

$$\int_0^t \sigma(X_s) dB_s,$$

qui correspond à la partie aléatoire, liée au mouvement brownien, et

$$\int_0^t b(X_s) ds,$$

qui correspond à la partie déterministe, liée à la dérive.

## 2 Solutions fortes et théorème d'existence-unicité

**Théorème 1.** *On suppose que  $X_0 \in L^2$  et que les fonctions  $b$  et  $\sigma$  sont lipschitziennes, c'est-à-dire qu'il existe une constante  $K > 0$  telle que, pour tous  $x, y \in \mathbb{R}$ ,*

$$|b(x) - b(y)| + |\sigma(x) - \sigma(y)| \leq K|x - y|.$$

*Alors l'équation différentielle stochastique admet une unique solution forte*

$$X \in M^2.$$

*Autrement dit, sous l'hypothèse de Lipschitz, il existe une solution, et cette solution est unique.*

### 2.1 Définition des espaces $M^2$ et $M^2([0, T])$

Avant de démontrer le théorème d'existence et d'unicité, on rappelle la définition des espaces dans lesquels on cherche les solutions de l'EDS.

Soit  $(\Omega, \mathcal{F}, \mathbb{P})$  un espace de probabilité muni d'une filtration  $(\mathcal{F}_t)_{t \geq 0}$ .

L'espace  $M^2([0, T])$  désigne l'ensemble des processus stochastiques  $(X_t)_{0 \leq t \leq T}$  qui sont adaptés à la filtration  $(\mathcal{F}_t)_{t \geq 0}$  et qui vérifient :

$$\mathbb{E} \left[ \int_0^T |X_t|^2 dt \right] < +\infty.$$

Autrement dit, un processus appartient à  $M^2([0, T])$  s'il est adapté et de carré intégrable sur l'intervalle  $[0, T]$ .

On munit cet espace de la norme :

$$\|X\|_{M^2([0, T])} = \left( \mathbb{E} \left[ \int_0^T |X_t|^2 dt \right] \right)^{1/2}.$$

Cette norme permet de mesurer la taille moyenne du processus sur l'intervalle  $[0, T]$ .

L'espace  $M^2$  désigne généralement l'espace des processus qui appartiennent à  $M^2([0, T])$  pour tout temps fini  $T > 0$ . Ainsi, dire que  $X \in M^2$  signifie que, pour tout  $T > 0$ ,

$$\mathbb{E} \left[ \int_0^T |X_t|^2 dt \right] < +\infty.$$

Une propriété importante de  $M^2([0, T])$  est que c'est un espace complet pour la norme précédente. Cela signifie que toute suite de Cauchy dans  $M^2([0, T])$  converge vers un élément de  $M^2([0, T])$ .

Cette propriété est essentielle dans la preuve d'existence par itération de Picard : elle permet de conclure que la suite des approximations  $(X^n)_{n \geq 0}$  converge vers un processus limite  $X$  appartenant encore à  $M^2([0, T])$ .

### 2.2 Preuve du théorème

Montrons l'unicité de la solution. On suppose que  $X$  et  $Y$  sont deux solutions de la même EDS, avec la même condition initiale :

$$X(0) = Y(0) = X_0.$$

Comme  $X$  et  $Y$  sont solutions, on a :

$$X(t) = X_0 + \int_0^t \sigma(X(s)) dB(s) + \int_0^t b(X(s)) ds,$$

et

$$Y(t) = X_0 + \int_0^t \sigma(Y(s)) dB(s) + \int_0^t b(Y(s)) ds.$$

On soustrait les deux égalités :

$$X(t) - Y(t) = \int_0^t [\sigma(X(s)) - \sigma(Y(s))] dB(s) + \int_0^t [b(X(s)) - b(Y(s))] ds.$$

L'objectif est de montrer que

$$X(t) - Y(t) = 0$$

pour tout  $t$ , donc que les deux solutions sont identiques.

On prend le carré puis l'espérance. On utilise l'inégalité :

$$(a + b)^2 \leq 2(a^2 + b^2).$$

On obtient :

$$\begin{aligned} \mathbb{E} [|X(t) - Y(t)|^2] &\leq 2\mathbb{E} \left[ \left( \int_0^t [\sigma(X(s)) - \sigma(Y(s))] dB_s \right)^2 \right] \\ &\quad + 2\mathbb{E} \left[ \left( \int_0^t [b(X(s)) - b(Y(s))] ds \right)^2 \right]. \end{aligned}$$

Pour le deuxième terme, on utilise l'inégalité de Cauchy-Schwarz :

$$\left( \int_0^t f(s) ds \right)^2 \leq t \int_0^t f(s)^2 ds.$$

Donc :

$$\mathbb{E} \left[ \left( \int_0^t [b(X(s)) - b(Y(s))] ds \right)^2 \right] \leq t \int_0^t \mathbb{E} [|b(X(s)) - b(Y(s))|^2] ds.$$

Pour le premier terme, on utilise l'isométrie d'Itô :

$$\mathbb{E} \left[ \left( \int_0^t H_s dB_s \right)^2 \right] = \mathbb{E} \left[ \int_0^t H_s^2 ds \right].$$

Ici,

$$H_s = \sigma(X(s)) - \sigma(Y(s)).$$

Donc :

$$\mathbb{E} \left[ \left( \int_0^t [\sigma(X(s)) - \sigma(Y(s))] dB(s) \right)^2 \right] = \int_0^t \mathbb{E} [|\sigma(X(s)) - \sigma(Y(s))|^2] ds.$$

Ensuite, comme  $b$  et  $\sigma$  sont lipschitziennes, on a :

$$|b(X(s)) - b(Y(s))|^2 \leq K^2 |X(s) - Y(s)|^2,$$

et

$$|\sigma(X(s)) - \sigma(Y(s))|^2 \leq K^2 |X(s) - Y(s)|^2.$$

On en déduit :

$$\mathbb{E} [|X(t) - Y(t)|^2] \leq 2K^2(t+1) \int_0^t \mathbb{E} [|X(s) - Y(s)|^2] ds.$$

Comme  $t \leq T$ , on peut écrire :

$$\mathbb{E} [|X(t) - Y(t)|^2] \leq 2K^2(T+1) \int_0^t \mathbb{E} [|X(s) - Y(s)|^2] ds.$$

On pose :

$$u(t) = \mathbb{E} [|X(t) - Y(t)|^2] .$$

Alors :

$$u(t) \leq C_T \int_0^t u(s) ds,$$

avec

$$C_T = 2(T+1)K^2.$$

Par le lemme de Gronwall, on obtient :

$$u(t) = 0.$$

Donc :

$$\mathbb{E} [|X(t) - Y(t)|^2] = 0.$$

Cela implique :

$$X(t) = Y(t)$$

presque sûrement pour tout  $t$ . Donc la solution est unique.

On montre l'existence par itération de Picard. On construit une suite de processus  $(X^n)_{n \geq 0}$  par le procédé d'itération de Picard.

On définit :

$$X^0(t) = X_0,$$

puis, pour tout  $n \geq 0$ ,

$$X^{n+1}(t) = X_0 + \int_0^t b(X^n(s)) ds + \int_0^t \sigma(X^n(s)) dB(s).$$

On part du processus constant  $X^0$ , puis on construit une première approximation  $X^1$ , puis une deuxième approximation  $X^2$ , etc.

On veut montrer que la suite  $(X^n)$  converge vers un processus  $X$ , et que ce processus  $X$  est solution de l'EDS.

On regarde la différence entre deux approximations successives :

$$X^{n+1}(t) - X^n(t) = \int_0^t [b(X^n(s)) - b(X^{n-1}(s))] ds + \int_0^t [\sigma(X^n(s)) - \sigma(X^{n-1}(s))] dB(s).$$

En utilisant les mêmes arguments que pour l'unicité, c'est-à-dire l'inégalité

$$(a + b)^2 \leq 2(a^2 + b^2),$$

l'inégalité de Cauchy-Schwarz, l'isométrie d'Itô et la condition de Lipschitz, on obtient :

$$\mathbb{E} [|X^{n+1}(t) - X^n(t)|^2] \leq C_T \int_0^t \mathbb{E} [|X^n(s) - X^{n-1}(s)|^2] ds,$$

où

$$C_T = 2(T + 1)K^2.$$

Ensuite, par récurrence, on montre qu'il existe une constante  $a > 0$  telle que :

$$\mathbb{E} [|X^{n+1}(t) - X^n(t)|^2] \leq a \frac{C_T^n t^n}{n!}.$$

Cette estimation est importante, car le terme

$$\frac{t^n}{n!}$$

tend très vite vers zéro lorsque  $n$  devient grand.

En particulier, sur  $[0, T]$ , on obtient :

$$\|X^{n+1} - X^n\|_{M^2([0, T])}^2 \leq a \frac{(C_T T)^n}{n!}.$$

Ainsi, la série

$$\sum_{n \geq 0} \|X^{n+1} - X^n\|_{M^2([0, T])}$$

converge. Cela montre que la suite  $(X^n)$  est une suite de Cauchy dans  $M^2([0, T])$ .

Comme l'espace  $M^2([0, T])$  est complet, il existe un processus

$$X \in M^2([0, T])$$

tel que

$$X^n \longrightarrow X$$

dans  $M^2([0, T])$ .

Il reste alors à passer à la limite dans la formule de Picard :

$$X^{n+1}(t) = X_0 + \int_0^t b(X^n(s)) ds + \int_0^t \sigma(X^n(s)) dB(s).$$

Grâce à la convergence de  $X^n$  vers  $X$ , et grâce au caractère lipschitzien de  $b$  et  $\sigma$ , on peut montrer que :

$$b(X^n) \longrightarrow b(X),$$

et

$$\sigma(X^n) \longrightarrow \sigma(X).$$

Donc, en passant à la limite, on obtient :

$$X(t) = X_0 + \int_0^t b(X(s)) ds + \int_0^t \sigma(X(s)) dB(s).$$

Ainsi,  $X$  est bien une solution de l'EDS. On a donc démontré l'existence.

On a montré l'unicité puis l'existence, donc l'EDS admet une unique solution forte dans  $M^2$  sous l'hypothèse de Lipschitz sur les coefficients  $b$  et  $\sigma$ .

### 3 Exemple simple d'EDS

Montrer que l'équation différentielle stochastique

$$dX(t) = rX(t)(K - X(t)) dt + \sigma X(t)(K - X(t)) dB(t)$$

avec

$$X(0) = x_0 \in ]0, K[,$$

admet une unique solution définie en tout temps, et qui vérifie  $X(t) \in [0, K]$  pour tout  $t \geq 0$ .

(On pourra modifier les coefficients en dehors de  $[0, K]$ , afin de se ramener à des coefficients globalement lipschitziens, et remarquer que la diffusion reste dans  $[0, K]$  en tout temps – puisque les coefficients s'annulent aux extrémités de l'intervalle.)

#### Résolution

On considère l'équation différentielle stochastique suivante, avec  $X(0) = x_0 \in ]0, K[$  :

$$dX(t) = r X(t)(K - X(t)) dt + \sigma X(t)(K - X(t)) dB(t)$$

Les coefficients  $b(x) = r x(K - x)$  et  $\sigma(x) = \sigma x(K - x)$  ne sont pas globalement lipschitziens sur  $\mathbb{R}$ . En effet, ils croissent comme  $x^2$  à l'infini, donc leur pente devient arbitrairement grande. On ne peut donc pas appliquer directement le théorème d'existence et unicité (théorème 5.1) qui demande des coefficients globalement lipschitziens.

Pour contourner ce problème, on modifie ces coefficients en dehors de l'intervalle  $[0, K]$ . On les garde inchangés sur  $[0, K]$ , mais on les rend constants (donc lipschitziens) à l'extérieur. Par exemple, on pose  $\pi(x) = \max(0, \min(K, x))$  (la projection sur  $[0, K]$ ), puis  $\tilde{b}(x) = r \pi(x)(K - \pi(x))$  et  $\tilde{\sigma}(x) = \sigma \pi(x)(K - \pi(x))$ . Ainsi, si  $x$  est entre 0 et  $K$ ,  $\tilde{b}(x) = b(x)$  et  $\tilde{\sigma}(x) = \sigma(x)$  ; si  $x$  est en dehors,  $\tilde{b}(x) = 0$  et  $\tilde{\sigma}(x) = 0$ . Ces nouvelles fonctions sont bornées et globalement lipschitziennes sur  $\mathbb{R}$ .

Le théorème 5.1 s'applique alors : il existe une unique solution forte globale  $\tilde{X}(t)$  pour l'équation modifiée, avec  $\tilde{X}(0) = x_0$ . Notre objectif est de montrer que cette solution  $\tilde{X}(t)$  reste en fait toujours dans l'intervalle  $[0, K]$ . Si on y arrive, alors sur toute la trajectoire on aura  $\tilde{b}(\tilde{X}(t)) = b(\tilde{X}(t))$  et  $\tilde{\sigma}(\tilde{X}(t)) = \sigma(\tilde{X}(t))$ , donc  $\tilde{X}$  sera aussi solution de l'équation originale.

Observons maintenant ce qui se passe aux bords. En  $x = 0$ , on a  $b(0) = 0$  et  $\sigma(0) = 0$ . Si le processus atteint 0, l'équation devient  $d\tilde{X}(t) = 0$ , donc il reste bloqué à 0 pour toujours. De même en  $x = K$  :  $b(K) = 0$  et  $\sigma(K) = 0$ , donc si le processus atteint  $K$ , il y reste indéfiniment.

Supposons par l'absurde que  $\tilde{X}(t)$  sorte un jour de  $[0, K]$ . Comme  $\tilde{X}$  est continu (ses trajectoires sont continues), pour sortir de l'intervalle il doit d'abord en toucher le bord. Soit  $\tau$  le premier instant où  $\tilde{X}(\tau)$  vaut 0 ou  $K$ . Considérons le cas où  $\tilde{X}(\tau) = 0$  (l'autre cas est symétrique). Par la propriété de Markov forte, le processus  $Y(t) = \tilde{X}(\tau + t)$  pour  $t \geq 0$  est une solution de la même équation (modifiée) avec condition initiale  $Y(0) = 0$ . Mais le processus constant  $Z(t) = 0$  est aussi une solution de cette équation avec la même condition initiale, car  $\tilde{b}(0) = 0$  et  $\tilde{\sigma}(0) = 0$ . Par unicité de la solution forte (garantie par le théorème 5.1), on doit avoir  $Y(t) = Z(t)$  pour tout  $t$ , c'est-à-dire  $\tilde{X}(\tau + t) = 0$  pour tout  $t \geq 0$ . Ainsi, une fois que le processus a touché 0, il y reste et ne peut pas en sortir. Le

même raisonnement s'applique si  $\tilde{X}(\tau) = K$ . Par conséquent, le processus ne peut jamais traverser le bord pour aller à l'extérieur de  $[0, K]$ . Il reste donc dans  $[0, K]$  pour tout  $t$ , presque sûrement.

Puisque  $\tilde{X}(t)$  est toujours dans  $[0, K]$ , et que sur cet intervalle les coefficients modifiés coïncident avec les coefficients originaux,  $\tilde{X}$  vérifie l'équation de départ :

$$\tilde{X}(t) = x_0 + \int_0^t b(\tilde{X}(s)) ds + \int_0^t \sigma(\tilde{X}(s)) dB(s)$$

C'est donc une solution globale de l'équation originale. L'unicité vient du fait que toute solution de l'originale reste dans  $[0, K]$  (même raisonnement) et coïncide donc avec la solution modifiée. En conclusion, l'équation différentielle stochastique donnée admet une unique solution  $X(t)$  définie pour tout  $t \geq 0$ , et cette solution vérifie  $X(t) \in [0, K]$  pour tout  $t$ , presque sûrement.

## 4 Convergence d'une EDS vers une EDO

On considère la famille d'équations différentielles stochastiques indexée par  $\varepsilon > 0$  :

$$dX_t^\varepsilon = b^\varepsilon(X_t^\varepsilon) dt + \sigma^\varepsilon(X_t^\varepsilon) dB_t, \quad X_0^\varepsilon = x, \quad (1)$$

où  $(B_t)_{t \geq 0}$  est un mouvement brownien standard, ainsi que l'équation différentielle ordinaire :

$$dx_t = b(x_t) dt, \quad x_0 = x. \quad (2)$$

On suppose les hypothèses suivantes :

**(H1)** Il existe une constante  $L > 0$  telle que, pour tous  $x, y \in \mathbb{R}$ ,

$$|b^\varepsilon(x) - b^\varepsilon(y)| \leq L|x - y| \quad \text{et} \quad |b(x) - b(y)| \leq L|x - y|.$$

**(H2)** Les dérivées convergent uniformément :  $\|b^\varepsilon - b\|_\infty \rightarrow 0$ .

**(H3)** Le coefficient de diffusion tend uniformément vers zéro :  $\|\sigma^\varepsilon\|_\infty \rightarrow 0$ .

**Théorème 4.1** (Convergence de l'EDS vers l'EDO). *Sous les hypothèses (H1), (H2) et (H3), si  $X^\varepsilon$  désigne la solution forte de (1) et  $(x_t)$  la solution de (2), alors pour tout  $T > 0$ ,*

$$\mathbb{E} \left[ \sup_{0 \leq t \leq T} |X_t^\varepsilon - x_t|^2 \right] \xrightarrow{\varepsilon \rightarrow 0} 0.$$

*Autrement dit, la solution de l'EDS converge vers la solution de l'EDO dans  $L^2$ , uniformément sur tout compact temporel.*

*Démonstration.* Par définition des solutions, on dispose des formes intégrales

$$X_t^\varepsilon = x + \int_0^t b^\varepsilon(X_s^\varepsilon) ds + \int_0^t \sigma^\varepsilon(X_s^\varepsilon) dB_s, \quad x_t = x + \int_0^t b(x_s) ds.$$

En soustrayant ces deux expressions :

$$X_t^\varepsilon - x_t = \int_0^t (b^\varepsilon(X_s^\varepsilon) - b(x_s)) ds + \int_0^t \sigma^\varepsilon(X_s^\varepsilon) dB_s.$$

On applique l'inégalité  $(a + b)^2 \leq 2a^2 + 2b^2$  et on pose

$$A_t = \left( \int_0^t (b^\varepsilon(X_s^\varepsilon) - b(x_s)) ds \right)^2, \quad M_t = \left( \int_0^t \sigma^\varepsilon(X_s^\varepsilon) dB_s \right)^2,$$

de sorte que  $|X_t^\varepsilon - x_t|^2 \leq 2A_t + 2M_t$ .

*Contrôle de  $A_t$ .* L'inégalité de Cauchy-Schwarz donne  $A_t \leq t \int_0^t |b^\varepsilon(X_s^\varepsilon) - b(x_s)|^2 ds$ . On décompose ensuite

$$b^\varepsilon(X_s^\varepsilon) - b(x_s) = (b^\varepsilon(X_s^\varepsilon) - b(X_s^\varepsilon)) + (b(X_s^\varepsilon) - b(x_s)),$$

où le premier terme est majoré par  $\|b^\varepsilon - b\|_\infty$  et le second par  $L|X_s^\varepsilon - x_s|$  grâce à (H1). Une nouvelle application de  $(a + b)^2 \leq 2a^2 + 2b^2$  donne :

$$A_t \leq 2t^2 \|b^\varepsilon - b\|_\infty^2 + 2L^2 t \int_0^t |X_s^\varepsilon - x_s|^2 ds.$$

*Contrôle de  $M_t$ .* Par l'isométrie d'Itô et l'hypothèse (H3) :

$$\mathbb{E}[M_t] = \mathbb{E} \left[ \int_0^t |\sigma^\varepsilon(X_s^\varepsilon)|^2 ds \right] \leq t \|\sigma^\varepsilon\|_\infty^2.$$

*Application du lemme de Gronwall.* En posant  $u(t) = \mathbb{E}[\sup_{0 \leq r \leq t} |X_r^\varepsilon - x_r|^2]$ , on rassemble les estimations précédentes : il existe une constante  $C > 0$  telle que

$$u(t) \leq C(\|b^\varepsilon - b\|_\infty^2 + \|\sigma^\varepsilon\|_\infty^2) + C \int_0^t u(s) ds.$$

Le lemme de Gronwall donne alors

$$u(t) \leq C(\|b^\varepsilon - b\|_\infty^2 + \|\sigma^\varepsilon\|_\infty^2) e^{Ct}.$$

Comme  $\|b^\varepsilon - b\|_\infty \rightarrow 0$  et  $\|\sigma^\varepsilon\|_\infty \rightarrow 0$  par (H2) et (H3), on conclut que  $u(t) \rightarrow 0$ , ce qui achève la démonstration.  $\square$

## 5 Dépendance en la condition initiale

On considère l'équation différentielle stochastique

$$dX_t = b(X_t) dt + \sigma(X_t) dB_t, \tag{3}$$

où les coefficients  $b$  et  $\sigma$  vérifient une condition de Lipschitz globale : il existe  $L > 0$  tel que, pour tous  $x, y \in \mathbb{R}$ ,

$$|b(x) - b(y)| \leq L|x - y| \quad \text{et} \quad |\sigma(x) - \sigma(y)| \leq L|x - y|.$$

On note  $(X_t^x)_{t \geq 0}$  et  $(X_t^y)_{t \geq 0}$  les solutions de (3) issues respectivement des conditions initiales  $X_0^x = x$  et  $X_0^y = y$ .

**Théorème 5.1** (Dépendance continue en la condition initiale). *Sous l'hypothèse de Lipschitz sur  $b$  et  $\sigma$ , pour tout  $T > 0$  il existe une constante  $C_T > 0$ , ne dépendant que de  $T$  et de  $L$ , telle que*

$$\mathbb{E} \left[ \sup_{0 \leq t \leq T} |X_t^x - X_t^y|^2 \right] \leq C_T |x - y|^2.$$

*En particulier, la solution de (3) dépend continûment de la condition initiale.*

*Démonstration.* On soustrait les formes intégrales des deux solutions :

$$X_t^x - X_t^y = (x - y) + \int_0^t (b(X_s^x) - b(X_s^y)) ds + \int_0^t (\sigma(X_s^x) - \sigma(X_s^y)) dB_s.$$

On pose  $D_t = X_t^x - X_t^y$ . L'inégalité  $(a + b + c)^2 \leq 3(a^2 + b^2 + c^2)$  donne

$$|D_t|^2 \leq 3|x - y|^2 + 3A_t + 3M_t,$$

avec

$$A_t = \left( \int_0^t (b(X_s^x) - b(X_s^y)) ds \right)^2, \quad M_t = \left( \int_0^t (\sigma(X_s^x) - \sigma(X_s^y)) dB_s \right)^2.$$

*Contrôle de  $A_t$ .* Par l'inégalité de Cauchy-Schwarz puis la condition de Lipschitz sur  $b$  :

$$A_t \leq t \int_0^t |b(X_s^x) - b(X_s^y)|^2 ds \leq L^2 t \int_0^t |D_s|^2 ds.$$

*Contrôle de  $M_t$ .* Par l'isométrie d'Itô puis la condition de Lipschitz sur  $\sigma$  :

$$\mathbb{E}[M_t] = \mathbb{E} \left[ \int_0^t |\sigma(X_s^x) - \sigma(X_s^y)|^2 ds \right] \leq L^2 \int_0^t \mathbb{E}[|D_s|^2] ds.$$

*Application du lemme de Gronwall.* En posant  $u(t) = \mathbb{E}[\sup_{0 \leq r \leq t} |D_r|^2]$ , on rassemble les estimations précédentes : il existe  $C > 0$  tel que

$$u(t) \leq 3|x - y|^2 + C \int_0^t u(s) ds.$$

Le lemme de Gronwall donne

$$u(t) \leq 3|x - y|^2 e^{Ct}.$$

En posant  $C_T = 3e^{CT}$ , on obtient bien, pour tout  $t \in [0, T]$ ,

$$\mathbb{E} \left[ \sup_{0 \leq t \leq T} |X_t^x - X_t^y|^2 \right] \leq C_T |x - y|^2. \quad \square$$

## 6 Simulation numérique par schéma d'Euler-Maruyama

Dans cette partie, on s'intéresse à la simulation numérique d'une équation différentielle stochastique de la forme

$$dX_t = b(X_t) dt + \sigma(X_t) dB_t, \quad X_0 = x.$$

Ici,  $(B_t)_{t \geq 0}$  désigne un mouvement brownien,  $b$  est le coefficient de dérive et  $\sigma$  est le coefficient de diffusion.

L'objectif est de construire une approximation numérique de la solution  $(X_t)_{t \in [0, T]}$  sur un intervalle de temps fixé  $[0, T]$ . Pour cela, on utilise le schéma d'Euler-Maruyama, qui est une adaptation du schéma d'Euler classique au cas des équations différentielles stochastiques.

## 6.1 Discrétisation de l'intervalle de temps

On découpe l'intervalle  $[0, T]$  en  $M$  sous-intervalles de même longueur. On note

$$0 = t_0 < t_1 < \dots < t_M = T.$$

Le pas de temps est donné par

$$h = \frac{T}{M}.$$

La solution continue  $(X_t)_{t \in [0, T]}$  est alors approchée par une suite de valeurs discrètes

$$X_0, X_1, \dots, X_M,$$

où  $X_k$  désigne une approximation de  $X_{t_k}$ .

## 6.2 Simulation du mouvement brownien

Le terme aléatoire de l'EDS provient du mouvement brownien. Pour simuler ce terme, on utilise les accroissements browniens

$$\Delta B_k = B_{t_{k+1}} - B_{t_k}.$$

Les accroissements du mouvement brownien sont indépendants et suivent une loi normale centrée de variance  $h$  :

$$\Delta B_k \sim \mathcal{N}(0, h).$$

Ainsi, en pratique, on peut simuler chaque accroissement par

$$\Delta B_k = \sqrt{h} Z_k,$$

où

$$Z_k \sim \mathcal{N}(0, 1).$$

## 6.3 Schéma d'Euler-Maruyama

On part de la forme intégrée de l'EDS :

$$X_t = x + \int_0^t b(X_s) ds + \int_0^t \sigma(X_s) dB_s.$$

Sur un petit intervalle  $[t_k, t_{k+1}]$ , on approxime les coefficients  $b(X_s)$  et  $\sigma(X_s)$  par leurs valeurs au début de l'intervalle, c'est-à-dire par  $b(X_k)$  et  $\sigma(X_k)$ . On obtient alors l'approximation suivante :

$$X_{k+1} = X_k + b(X_k)h + \sigma(X_k)\Delta B_k.$$

Ce schéma est appelé schéma d'Euler-Maruyama. Il s'écrit donc :

$$\boxed{X_{k+1} = X_k + b(X_k)h + \sigma(X_k)\Delta B_k}$$

avec

$$\Delta B_k \sim \mathcal{N}(0, h).$$

Le terme

$$b(X_k)h$$

correspond à la partie déterministe de l'évolution, tandis que le terme

$$\sigma(X_k)\Delta B_k$$

correspond à la partie aléatoire due au mouvement brownien.

## 6.4 Programme Python utilisé

Le programme écrit en Python prend en entrée les paramètres suivants :

$$(\sigma, b, x, T, N),$$

où  $x$  est la condition initiale,  $T$  l'horizon de temps et  $N$  le nombre de trajectoires simulées.

La fonction renvoie la grille de temps ainsi qu'un tableau contenant les  $N$  trajectoires simulées. Chaque ligne du tableau correspond à une trajectoire, et chaque colonne correspond à un instant de la grille de temps.

Le schéma d'Euler-Maruyama est implémenté dans la ligne suivante :

$$X[i, k + 1] = X[i, k] + b(X[i, k])h + \sigma(X[i, k])\Delta B_k.$$

En Python, cette étape s'écrit sous la forme :

$$X[i, k + 1] = x\_actuel + derive * h + diffusion * delta\_B.$$

## 6.5 Gestion des difficultés numériques

Certains coefficients de diffusion peuvent poser des difficultés lors de la simulation. Par exemple, si

$$\sigma(x) = \sqrt{x},$$

alors le coefficient de diffusion n'est défini que pour  $x \geq 0$ .

Cependant, le schéma d'Euler-Maruyama peut produire numériquement une valeur négative de  $X_k$ , même lorsque le modèle théorique est censé rester positif. Dans ce cas, le calcul de  $\sqrt{X_k}$  n'est plus possible.

Pour éviter ce problème, on utilise la partie positive :

$$X_k^+ = \max(X_k, 0).$$

La troncature complète évalue les coefficients en la partie positive, conserve un état interne signé et affiche sa partie positive. La section numérique précise la mise à jour.

Une autre difficulté apparaît dans le cas du diffusion à racine carrée avec Euler réfléchi, de la forme

$$dX_t = \sigma\sqrt{X_t} dB_t + \delta dt.$$

Dans ce cas, pour éviter les problèmes numériques liés aux valeurs négatives, on remplace  $X_k$  par sa valeur absolue dans le coefficient de diffusion. Autrement dit, on utilise

$$\sqrt{|X_k|}$$

à la place de

$$\sqrt{X_k}.$$

Cette correction permet de poursuivre la simulation même lorsque le schéma numérique produit temporairement une valeur négative.

## 6.6 Résumé de la méthode

La méthode utilisée peut être résumée de la manière suivante :

$$\begin{cases} X_0 = x, \\ \Delta B_k = \sqrt{h}Z_k, \quad Z_k \sim \mathcal{N}(0, 1), \\ X_{k+1} = X_k + b(X_k)h + \sigma(X_k)\Delta B_k. \end{cases}$$

Ainsi, le schéma d'Euler-Maruyama permet de simuler numériquement plusieurs trajectoires d'une EDS en combinant une partie déterministe, donnée par la dérive  $b$ , et une partie aléatoire, donnée par le coefficient de diffusion  $\sigma$  et le mouvement brownien.

## 7 Résultats numériques

Dans cette partie, nous présentons les résultats obtenus à partir des simulations numériques réalisées en Python. L'objectif est d'illustrer le comportement des trajectoires simulées par le schéma d'Euler-Maruyama, puis d'étudier certains cas particuliers où des difficultés numériques peuvent apparaître.

Nous étudions également la convergence d'une EDS vers une EDO lorsque le coefficient de diffusion tend vers zéro, ainsi que la comparaison entre le schéma d'Euler-Maruyama et le schéma de Milstein.

### 7.1 Simulation du mouvement brownien

Avant de simuler une équation différentielle stochastique, nous avons d'abord simulé un mouvement brownien  $(B_t)_{t \in [0, T]}$ . Le mouvement brownien représente le bruit aléatoire qui intervient dans l'EDS.

Les accroissements du mouvement brownien sont simulés par

$$\Delta B_k = \sqrt{h}Z_k,$$

où

$$Z_k \sim \mathcal{N}(0, 1).$$

La trajectoire obtenue est continue mais très irrégulière. Cela illustre le caractère aléatoire du bruit brownien.

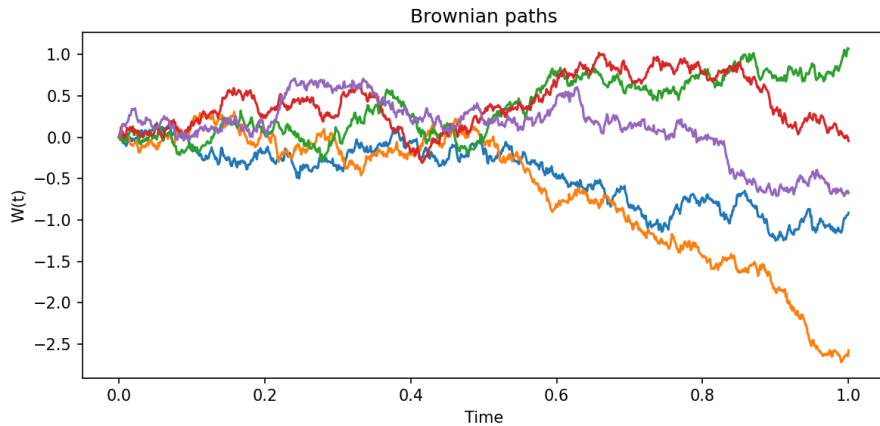


Figure 1: Simulation d'un mouvement brownien sur l'intervalle  $[0, T]$ .

On observe que la trajectoire fluctue de manière irrégulière autour de zéro. Cette trajectoire servira ensuite à construire les trajectoires approchées de l'EDS.

## 7.2 Simulation d'une EDS simple

Nous considérons d'abord une EDS simple de la forme

$$dX_t = \mu dt + \sigma dB_t,$$

où  $\mu$  et  $\sigma$  sont des constantes.

Dans ce cas, le coefficient de dérive est donné par

$$b(x) = \mu,$$

et le coefficient de diffusion est donné par

$$\sigma(x) = \sigma.$$

Le schéma d'Euler-Maruyama devient alors

$$X_{k+1} = X_k + \mu h + \sigma \Delta B_k.$$

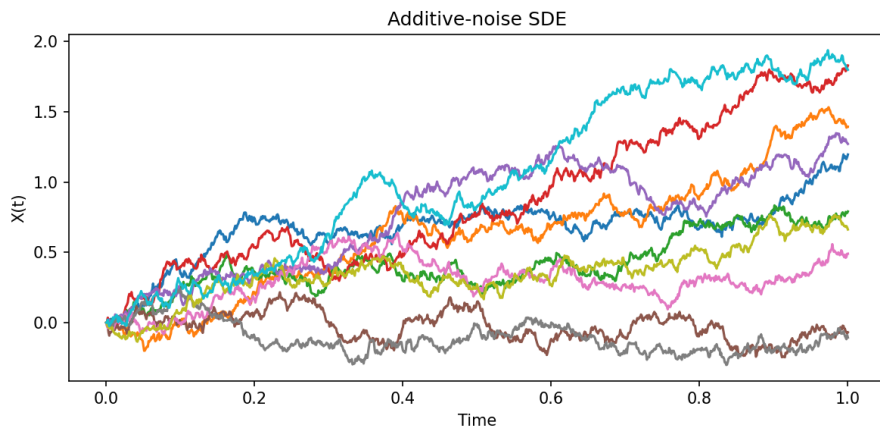


Figure 2: Simulation de plusieurs trajectoires d'une EDS simple.

Lorsque  $\mu > 0$ , les trajectoires ont globalement tendance à augmenter. Cependant, elles restent perturbées par le terme aléatoire  $\sigma dB_t$ . On remarque également que plus le coefficient de diffusion est grand, plus les trajectoires sont dispersées.

## 7.3 Cas d'un coefficient de diffusion non lipschitzien : $\sigma(x) = \sqrt{x}$

Nous étudions ensuite le cas

$$\sigma(x) = \sqrt{x}.$$

Ce cas est intéressant car la fonction  $x \mapsto \sqrt{x}$  n'est pas lipschitzienne en zéro. De plus, elle n'est définie que pour  $x \geq 0$ . Or, dans une simulation numérique, le schéma d'Euler-Maruyama explicite peut produire temporairement des valeurs négatives, même si la solution théorique est censée rester positive.

En effet, le schéma explicite classique s'écrit

$$X_{k+1} = X_k + b(X_k)h + \sqrt{X_k}\Delta B_k.$$

À cause du terme aléatoire  $\Delta B_k$ , il est possible que la valeur approchée  $X_{k+1}$  devienne négative. Dans ce cas, le calcul de la racine carrée au pas suivant n'est plus possible.

Une première correction consisterait à remplacer les valeurs négatives par zéro. Cependant, cette méthode peut provoquer un effet artificiel d'écrasement en zéro : certaines trajectoires peuvent rester bloquées près de zéro à cause du schéma numérique.

La version actuelle utilise Euler avec troncature complète. L'état interne  $Y_k$  reste signé ; les coefficients sont évalués en  $Y_k^+ = \max(Y_k, 0)$  :

$$Y_{k+1} = Y_k + b(Y_k^+)h + \sqrt{Y_k^+}\Delta B_k, \quad X_k = Y_k^+.$$

La positivité de la sortie ne constitue pas une simulation exacte. Ce schéma conserve un biais de discrétisation qui doit être contrôlé en raffinant la grille.

L'ancienne mise à jour semi-implicite fondée sur la racine positive d'une équation quadratique a été retirée : elle introduisait une dérive parasite. Le test de régression sur  $dX_t = \sqrt{X_t}dB_t$ ,  $X_0 = 1$ , contrôle que l'espérance simulée reste proche de 1 au lieu d'environ 1,5 à  $T = 1$ .

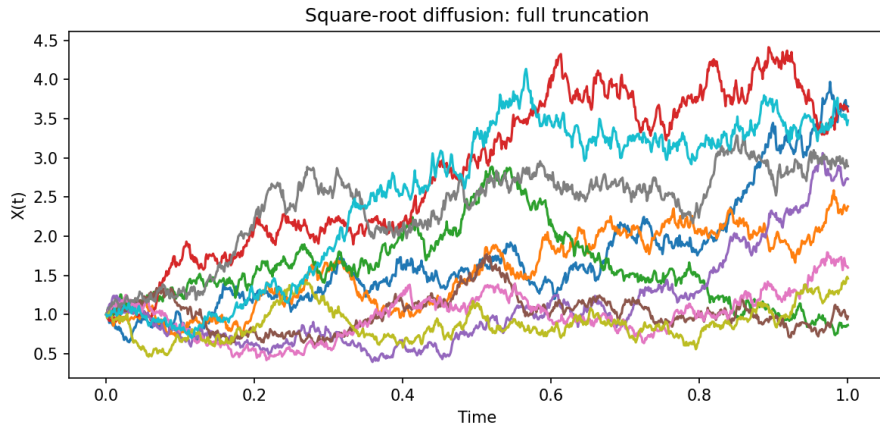


Figure 3: Diffusion en racine carrée : Euler avec troncature complète, graine 42.

## 7.4 Diffusion à racine carrée avec Euler réfléchi

Nous avons également simulé un modèle à diffusion en racine carrée et dérive constante :

$$dX_t = \sigma\sqrt{X_t}dB_t + \delta dt.$$

Ce modèle ne possède pas la dérive de retour à la moyenne  $\kappa(\theta - X_t)$  du CIR financier classique. La réflexion ci-dessous est une correction numérique illustrative, sans preuve de convergence vers une EDS réfléchie.

Dans ce modèle, le coefficient de diffusion dépend de  $\sqrt{X_t}$ . Comme précédemment, le schéma numérique peut produire des valeurs négatives. Pour éviter que le coefficient de diffusion ne soit pas défini, nous utilisons une version réfléchie en zéro.

Concrètement, on remplace

$$\sqrt{X_k}$$

par

$$\sqrt{|X_k|}.$$

Le schéma numérique utilisé est donc de la forme

$$X_{k+1} = \left| X_k + \delta h + \sigma \sqrt{X_k} \Delta B_k \right|.$$

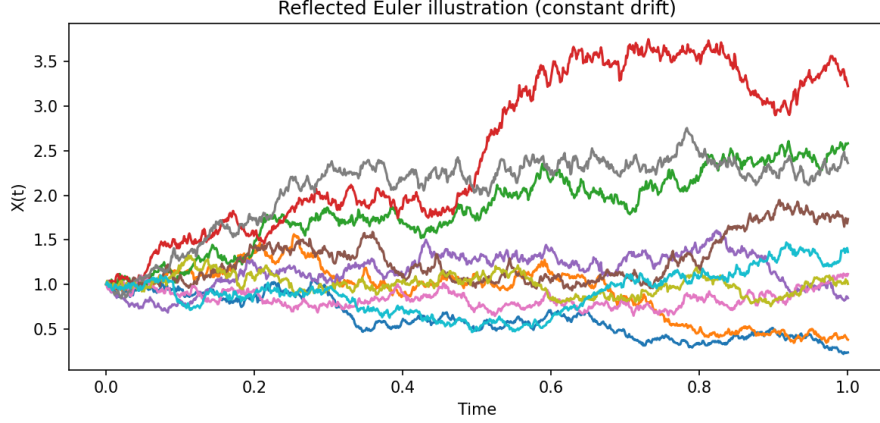


Figure 4: Simulation de la diffusion à racine carrée avec Euler réfléchi.

Cette méthode permet de poursuivre la simulation même lorsque le schéma produit une valeur négative. La réflexion en zéro est donc une correction numérique qui permet d'obtenir des trajectoires exploitables.

## 7.5 Convergence de l'EDS vers l'EDO lorsque le coefficient de diffusion tend vers zéro

Nous avons ensuite étudié la convergence d'une EDS vers une EDO lorsque le coefficient de diffusion tend uniformément vers zéro.

On considère une famille d'EDS indexée par un paramètre  $\varepsilon > 0$  :

$$dX_t^\varepsilon = b(X_t^\varepsilon) dt + \varepsilon \sigma(X_t^\varepsilon) dB_t.$$

Lorsque  $\varepsilon$  tend vers zéro, le coefficient de diffusion

$$\sigma_\varepsilon(x) = \varepsilon \sigma(x)$$

tend uniformément vers zéro, à condition que  $\sigma$  soit bornée.

L'EDS se rapproche alors de l'EDO déterministe :

$$dx_t = b(x_t) dt.$$

Pour quantifier cette convergence, nous avons calculé l'erreur uniforme

$$\sup_{0 \leq t \leq T} |X_t^\varepsilon - x_t|,$$

où  $(x_t)_{t \in [0, T]}$  désigne la solution de l'EDO.

La comparaison numérique utilise une référence EDO exacte et des erreurs sur la grille. La somme des valeurs absolues des incréments browniens ne fournit pas une borne continue uniforme exploitable lorsque la grille est raffinée ; cette ancienne illustration a été retirée.

Dans un premier temps, nous avons représenté les trajectoires simulées pour différentes valeurs de  $\varepsilon$ , ainsi que la solution de l'EDO associée.

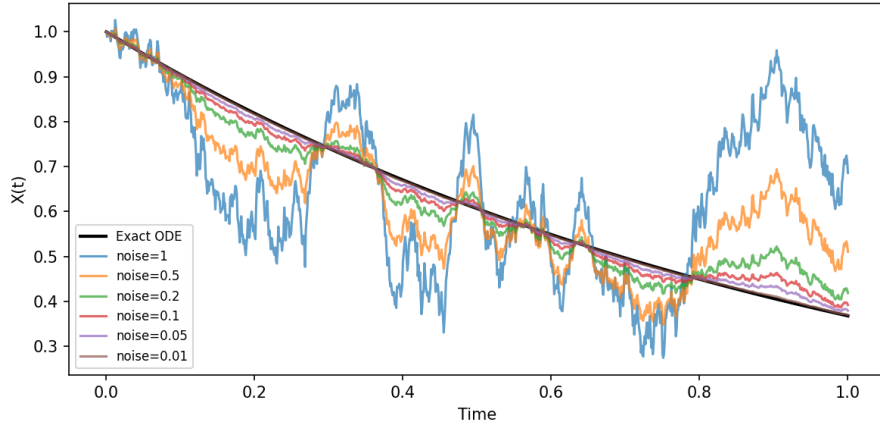


Figure 5: Convergence des trajectoires de l'EDS vers la solution de l'EDO lorsque  $\varepsilon \rightarrow 0$ .

Les résultats numériques montrent que lorsque  $\varepsilon$  diminue, les trajectoires de l'EDS se rapprochent de la solution de l'EDO. Cela illustre le fait que la disparition progressive du coefficient de diffusion réduit l'influence du bruit brownien.

Afin de quantifier plus précisément cette convergence, nous avons ensuite calculé, pour chaque valeur de  $\varepsilon$ , l'erreur moyenne

$$\mathbb{E} \left[ \sup_{0 \leq t \leq T} |X_t^\varepsilon - x_t| \right].$$

En pratique, cette espérance est approchée par une moyenne empirique sur les  $N$  trajectoires simulées. Autrement dit, si  $X^{\varepsilon,i}$  désigne la  $i$ -ème trajectoire simulée, l'erreur moyenne est approximée par

$$\frac{1}{N} \sum_{i=1}^N \sup_{0 \leq t \leq T} |X_t^{\varepsilon,i} - x_t|.$$

La version reproductible utilise la solution exacte de l'EDO,  $x(t) = e^{-t}$ , et 1 000 trajectoires avec une grille fixe de 1 000 pas. Les mêmes incréments browniens sont utilisés pour les différentes amplitudes du bruit. L'erreur est un maximum sur la grille, et non un supremum continu. Les barres représentent 1,96 fois l'erreur standard Monte-Carlo de la moyenne. Elles ne constituent pas une borne théorique de convergence. Le plancher d'erreur comprend le biais de discrétisation temporelle.

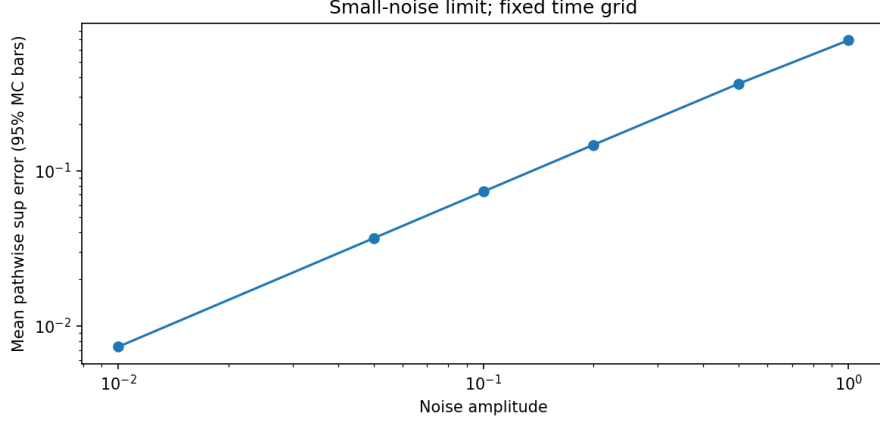


Figure 6: Erreur moyenne sur la grille et intervalles Monte-Carlo approximatifs à 95%.

Cette expérience illustre la diminution de l'erreur lorsque le bruit diminue à grille fixe ; elle ne remplace pas une preuve de convergence ni une étude du passage  $h \rightarrow 0$ .

## 7.6 Comparaison des schémas d'Euler-Maruyama et de Milstein

Enfin, nous avons comparé le schéma d'Euler-Maruyama avec le schéma de Milstein. L'objectif est d'étudier numériquement l'ordre de convergence de ces deux méthodes.

On considère une EDS générale de la forme

$$dX_t = b(X_t) dt + \sigma(X_t) dB_t.$$

Le schéma d'Euler-Maruyama associé est donné par

$$X_{k+1} = X_k + b(X_k)h + \sigma(X_k)\Delta B_k,$$

où

$$\Delta B_k = B_{t_{k+1}} - B_{t_k} \sim \mathcal{N}(0, h).$$

Le schéma de Milstein est une amélioration du schéma d'Euler-Maruyama. Il ajoute un terme correctif qui fait intervenir la dérivée du coefficient de diffusion. Lorsque  $\sigma$  est dérivable, le schéma de Milstein s'écrit :

$$X_{k+1} = X_k + b(X_k)h + \sigma(X_k)\Delta B_k + \frac{1}{2}\sigma(X_k)\sigma'(X_k)((\Delta B_k)^2 - h).$$

Le terme supplémentaire

$$\frac{1}{2}\sigma(X_k)\sigma'(X_k)((\Delta B_k)^2 - h)$$

permet d'améliorer la précision de l'approximation. C'est pourquoi le schéma de Milstein possède en général un meilleur ordre de convergence forte que le schéma d'Euler-Maruyama.

Pour comparer les deux schémas, nous avons choisi une EDS dont la solution exacte est connue : le mouvement brownien géométrique.

On considère l'EDS

$$dX_t = \mu X_t dt + \sigma X_t dB_t, \quad X_0 = x_0.$$

Dans ce cas, on a

$$b(x) = \mu x$$

et

$$\sigma(x) = \sigma x.$$

La dérivée du coefficient de diffusion est donc

$$\sigma'(x) = \sigma.$$

La solution exacte est donnée par

$$X_t = x_0 \exp \left( \left( \mu - \frac{\sigma^2}{2} \right) t + \sigma B_t \right).$$

Pour cette EDS, le schéma d'Euler-Maruyama devient

$$X_{k+1} = X_k + \mu X_k h + \sigma X_k \Delta B_k.$$

En appliquant la formule générale du schéma de Milstein, on obtient

$$X_{k+1} = X_k + \mu X_k h + \sigma X_k \Delta B_k + \frac{1}{2} \sigma^2 X_k ((\Delta B_k)^2 - h).$$

La différence entre les deux schémas vient donc du terme correctif

$$\frac{1}{2} \sigma^2 X_k ((\Delta B_k)^2 - h).$$

Pour comparer les deux méthodes, nous calculons l'erreur au temps final :

$$|X_T - X_T^{\text{approx}}|,$$

où  $X_T$  désigne la solution exacte et  $X_T^{\text{approx}}$  la valeur approchée obtenue par Euler-Maruyama ou par Milstein.

Cette erreur est ensuite moyennée sur plusieurs trajectoires simulées. On répète cette expérience pour plusieurs valeurs du pas de temps  $h$ , puis on trace l'erreur en fonction de  $h$  en échelle logarithmique.

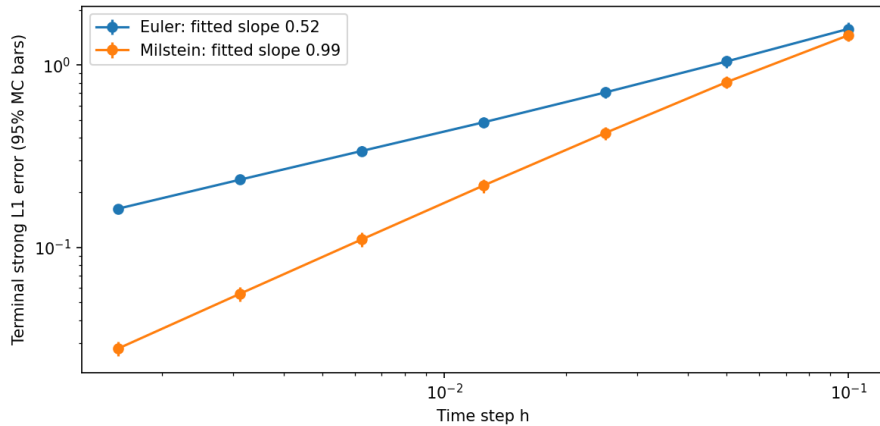


Figure 7: Comparaison de la convergence des schémas d'Euler-Maruyama et de Milstein en échelle log-log.

Sur un graphique en échelle log-log, si l'erreur se comporte comme

$$\text{erreur}(h) \simeq Ch^\alpha,$$

alors on a

$$\log(\text{erreur}(h)) \simeq \log(C) + \alpha \log(h).$$

La pente ajustée est une estimation empirique ; elle dépend des grilles et de l'incertitude Monte-Carlo. La version actuelle couple toutes les grilles au même Brownien fin et ajuste les quatre grilles les plus fines.

Numériquement, on s'attend à observer une pente proche de  $1/2$  pour le schéma d'Euler-Maruyama et une pente proche de  $1$  pour le schéma de Milstein. Avec la graine 42 et 5 000 trajectoires, les pentes ajustées valent environ 0,523 pour Euler et 0,988 pour Milstein. Les barres d'erreur représentent des intervalles Monte-Carlo approximatifs pour les erreurs moyennes, sans être des intervalles de confiance sur les pentes.

## 7.7 Conclusion des résultats numériques

Les simulations numériques montrent que le schéma d'Euler-Maruyama permet d'approximer simplement les trajectoires d'une équation différentielle stochastique. La simulation du mouvement brownien permet d'abord de visualiser le bruit aléatoire intervenant dans l'EDS.

Les exemples étudiés mettent aussi en évidence certaines difficultés numériques. En particulier, lorsque le coefficient de diffusion est donné par  $\sigma(x) = \sqrt{x}$ , le schéma peut produire des valeurs négatives. Il est alors nécessaire d'utiliser une correction, comme la partie positive ou la réflexion en zéro.

Les simulations montrent également que lorsque le coefficient de diffusion tend uniformément vers zéro, les trajectoires de l'EDS se rapprochent de la solution de l'EDO associée. Enfin, la comparaison entre les schémas d'Euler-Maruyama et de Milstein montre que le schéma de Milstein possède un meilleur ordre de convergence dans le cas du mouvement brownien géométrique.

## Conclusion

Ce projet a permis de mieux comprendre les équations différentielles stochastiques, aussi bien sur le plan théorique que numérique. La démonstration du théorème d'existence et d'unicité par itération de Picard est assez technique mais elle repose sur des arguments relativement naturels une fois qu'on a bien en tête la structure de l'espace  $M^2$ . Les résultats de stabilité : convergence vers une EDO et dépendance en la condition initiale, s'obtiennent tous les deux par le même outil, le lemme de Gronwall.

Du côté numérique, le schéma d'Euler-Maruyama est simple à coder mais il faut faire attention à certains cas particuliers. Quand le coefficient de diffusion est par exemple  $\sigma(x) = \sqrt{x}$ , le schéma peut produire des valeurs négatives et il faut prévoir une correction. La comparaison avec Milstein montre clairement que ce dernier converge plus vite (ordre 1 contre  $1/2$ ), même si la différence n'est pas toujours énorme en pratique pour des pas de temps raisonnables.

Les simulations illustrent les résultats sur les exemples testés. Elles ne constituent pas une validation générale, en particulier pour les corrections numériques au voisinage de zéro.

# Bibliographie

- F. Comets et T. Meyre, *Calcul stochastique et modèles de diffusions*, Dunod.
- R. Lord, R. Koekkoek et D. van Dijk, *A Comparison of Biased Simulation Schemes for Stochastic Volatility Models*, 2008.
- E. Altman, *Cours de calcul stochastique*.

## Annexe : code Python et exécutable

Le dépôt fournit le script des expériences et le module de calcul. La capture suivante provient du travail d'origine ; l'exécutable et son interface ne sont pas distribués avec cette version. Les figures numériques du rapport ont été régénérées avec la graine 42.

Nous présentons également une capture de l'interface de l'exécutable, afin de montrer à quoi ressemble l'application une fois lancée.

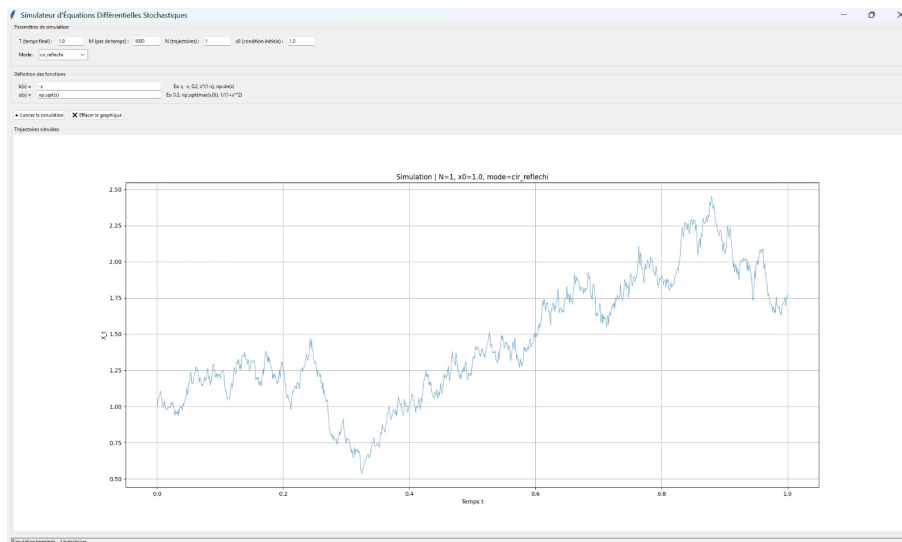


Figure 8: Interface de l'exécutable utilisé pour lancer les simulations numériques.

```
1 """Reproducible experiments. Example: python code_PNI.py --seed 42 --
   paths 5000."""
2 import argparse
3 import csv
4 import json
5 from pathlib import Path
6 import matplotlib
7 matplotlib.use("Agg")
8 import matplotlib.pyplot as plt
9 import numpy as np
10 from simulation import brownian, euler_eds, gbm_convergence
11
12
13 def main():
14     parser = argparse.ArgumentParser(description=__doc__)
15     parser.add_argument("--seed", type=int, default=42)
16     parser.add_argument("--paths", type=int, default=5000)
```

```

17 parser.add_argument("--output", type=Path, default=Path("results"))
18 args = parser.parse_args()
19 args.output.mkdir(parents=True, exist_ok=True)
20 rng = np.random.default_rng(args.seed)
21 def save(name):
22     plt.tight_layout()
23     plt.savefig(args.output/name, dpi=150)
24     plt.close()
25 t, b = brownian(paths=5, rng=rng)
26 plt.figure(figsize=(8, 4)); plt.plot(t, b.T)
27 plt.xlabel("Time"); plt.ylabel("W(t)"); plt.title("Brownian paths")
28 save("figure_brownien.png")
29 experiments = [
30     (lambda x: np.full_like(x, .5), lambda x: np.ones_like(x), 0., "
      standard", "figure_eds_simple.png", "Additive-noise SDE"),
31     (np.sqrt, lambda x: np.full_like(x, .1), 1., "full_truncation",
      "figure_sigma_racine.png", "Square-root diffusion: full
      truncation"),
32     (lambda x: .7*np.sqrt(x), lambda x: np.full_like(x, .3), 1., "
      reflection", "figure_cir_reflechi.png", "Reflected Euler
      illustration (constant drift)")]
33 for sigma, drift, x0, mode, filename, title in experiments:
34     t, x = euler_eds(sigma, drift, x0, 1., 10, mode=mode, rng=rng)
35     plt.figure(figsize=(8, 4)); plt.plot(t, x.T)
36     plt.xlabel("Time"); plt.ylabel("X(t)"); plt.title(title)
37     save(filename)
38 # Coupled draws across noise amplitudes; exact ODE reference exp(-t)
39
40 epsilons = np.array([1., .5, .2, .1, .05, .01])
41 increments = rng.normal(size=(1000, 1000))*np.sqrt(.001)
42 t = np.linspace(0, 1, 1001); exact_ode = np.exp(-t)
43 errors, ses = [], []
44 plt.figure(figsize=(8, 4)); plt.plot(t, exact_ode, color="black", lw
    =2, label="Exact ODE")
45 for eps in epsilons:
46     x = np.ones((1000, 1001))
47     for k in range(1000):
48         state = x[:, k]
49         x[:, k+1] = state-state*.001+eps/(1+state*state)*increments
50         [:, k]
51     error = np.max(np.abs(x-exact_ode), axis=1)
52     errors.append(float(error.mean())); ses.append(float(error.std(
53         ddof=1)/np.sqrt(len(error))))
54     plt.plot(t, x[0], alpha=.7, label=f"noise={eps:g}")
55 plt.xlabel("Time"); plt.ylabel("X(t)"); plt.legend(fontsize=8)
56 save("figure_convergence_edo.png")
57 plt.figure(figsize=(8, 4)); plt.errorbar(epsilons, errors, yerr
    =1.96*np.array(ses), marker="o")
58 plt.xscale("log"); plt.yscale("log"); plt.xlabel("Noise amplitude")
59 plt.ylabel("Mean pathwise sup error (95% MC bars)")
60 plt.title("Small-noise limit; fixed time grid")
61 save("figure_quantification_convergence.png")
62 with (args.output/"small_noise.csv").open("w", newline="") as f:
63     writer=csv.writer(f); writer.writerow(["epsilon", "
64         mean_sup_error", "mc_standard_error"])
65     writer.writerows(zip(epsilons, errors, ses))
66 result = gbm_convergence(paths=args.paths, seed=args.seed)
67 with (args.output/"convergence.csv").open("w", newline="") as f:

```

```

64     writer=csv.writer(f); writer.writerow(["steps","h","
        euler_l1_error","euler_mc_se","milstein_l1_error","
        milstein_mc_se"])
65     writer.writerows(zip(result.steps,result.h,result.euler_error,
        result.euler_se,result.milstein_error,result.milstein_se))
66 plt.figure(figsize=(8, 4))
67 for label, error, se, slope in [("Euler",result.euler_error,result.
    euler_se,result.euler_slope),("Milstein",result.milstein_error,
    result.milstein_se,result.milstein_slope)]:
68     plt.errorbar(result.h,error,yerr=1.96*se,marker="o",label=f"{
        label}: fitted slope {slope:.2f}")
69 plt.xscale("log"); plt.yscale("log"); plt.xlabel("Time step h")
70 plt.ylabel("Terminal strong L1 error (95% MC bars)"); plt.legend()
71 save("figure_euler_milstein_loglog.png")
72 summary={"seed":args.seed,"paths":args.paths,"mu":2.,"volatility"
    :1.,"T":1.,"x0":1.,"euler_slope":result.euler_slope,"
    milstein_slope":result.milstein_slope,"slope_fit":"finest four
    grids", "numpy":np.__version__,"matplotlib":matplotlib.
    __version__}
73 (args.output/"summary.json").write_text(json.dumps(summary, indent
    =2)+"\n")
74 print(json.dumps(summary, indent=2))
75
76
77 if __name__ == "__main__":
78     main()

```

```

1  """Reusable SDE simulations; importing this module never launches
    experiments."""
2  from dataclasses import dataclass
3  import numpy as np
4
5
6  def _grid(T, steps, paths):
7      if not np.isfinite(T) or T <= 0 or not isinstance(steps, (int, np.
        integer)) or steps < 1 or not isinstance(paths, (int, np.integer)
        ) or paths < 1:
8          raise ValueError("Positive finite horizon and positive integer
        steps/paths required")
9
10
11 def brownian(T=1.0, steps=1000, paths=1, rng=None):
12     _grid(T, steps, paths)
13     rng = np.random.default_rng() if rng is None else rng
14     increments = rng.normal(size=(paths, steps)) * np.sqrt(T / steps)
15     return np.linspace(0, T, steps + 1), np.column_stack((np.zeros(paths
        ), np.cumsum(increments, axis=1)))
16
17
18 def euler_eds(sigma, b, x0, T, N, M=1000, mode="standard", rng=None):
19     """Vectorized Euler-Maruyama, full truncation, or reflected Euler.
20
21     Full truncation keeps the signed internal Euler state, evaluates
    both
22     coefficients at max(state, 0), and returns its positive part. This
    is a
23     biased finite-step approximation, not an exact nonnegative
    simulation.

```

```

24     Reflection is a separate numerical illustration, not a financial CIR
        model.
25     """
26     _grid(T, M, N)
27     if not np.isfinite(x0) or (mode != "standard" and x0 < 0):
28         raise ValueError("Finite initial state (nonnegative for square-
            root modes) required")
29     if mode not in {"standard", "full_truncation", "reflection"}:
30         raise ValueError("Unknown mode")
31     rng = np.random.default_rng() if rng is None else rng
32     h = T / M
33     X = np.empty((N, M + 1))
34     X[:, 0] = x0
35     raw = np.full(N, float(x0))
36     for k in range(M):
37         z = rng.normal(size=N) * np.sqrt(h)
38         evaluated = raw if mode == "standard" else np.maximum(raw, 0)
39         raw = raw + b(evaluated) * h + sigma(evaluated) * z
40         if mode == "reflection":
41             raw = np.abs(raw)
42         if not np.isfinite(raw).all():
43             raise FloatingPointError("Non-finite trajectory: check
                coefficients and time step")
44         X[:, k + 1] = np.maximum(raw, 0) if mode == "full_truncation"
            else raw
45     return np.linspace(0, T, M + 1), X
46
47
48 @dataclass
49 class Convergence:
50     steps: np.ndarray
51     h: np.ndarray
52     euler_error: np.ndarray
53     milstein_error: np.ndarray
54     euler_se: np.ndarray
55     milstein_se: np.ndarray
56     euler_slope: float
57     milstein_slope: float
58
59
60 def gbm_convergence(steps=(10, 20, 40, 80, 160, 320, 640), paths=5000,
61                     seed=42, mu=2.0, volatility=1.0, x0=1.0, T=1.0):
62     """Terminal strong L1 errors against the exact GBM on coupled
        Brownian paths.
63
64     All grids share the finest Brownian increments. Standard errors
        quantify
65     Monte-Carlo uncertainty of each mean; slope fits remain empirical
        estimates.
66     """
67     steps = np.asarray(steps)
68     if steps.ndim != 1 or len(steps) < 2 or not np.issubdtype(steps.
        dtype, np.integer) or np.any(steps < 1) or np.any(np.diff(steps)
        <= 0):
69         raise ValueError("At least two strictly increasing positive
            integer grids required")
70     _grid(T, int(steps[-1]), paths)
71     if paths < 2 or not np.isfinite([mu, volatility, x0]).all() or

```

```

72     volatility < 0 or x0 <= 0:
73         raise ValueError("At least two paths, finite parameters,
74             positive x0, nonnegative volatility required")
75     finest = int(steps[-1])
76     if np.any(finest % steps):
77         raise ValueError("Each grid must divide the finest grid")
78     rng = np.random.default_rng(seed)
79     fine = rng.normal(size=(paths, finest)) * np.sqrt(T / finest)
80     exact = x0 * np.exp((mu - 0.5 * volatility**2)*T + volatility*fine.
81         sum(axis=1))
82     errors_e, errors_m, ses_e, ses_m = [], [], [], []
83     for n in steps:
84         h = T / n
85         increments = fine.reshape(paths, int(n), finest // int(n)).sum(
86             axis=2)
87         euler = np.full(paths, x0)
88         milstein = np.full(paths, x0)
89         for k in range(n):
90             z = increments[:, k]
91             euler *= 1 + mu*h + volatility*z
92             milstein *= 1 + mu*h + volatility*z + 0.5*volatility**2*(z*z
93                 -h)
94         for approx, errors, ses in [(euler, errors_e, ses_e), (milstein,
95             errors_m, ses_m)]:
96             error = np.abs(exact - approx)
97             errors.append(error.mean())
98             ses.append(error.std(ddof=1)/np.sqrt(paths))
99     h = T/steps
100     # Fit the finest four grids to reduce pre-asymptotic coarse-grid
101     effects.
102     tail = slice(-min(4, len(steps)), None)
103     return Convergence(steps, h, np.array(errors_e), np.array(errors_m),
104         np.array(ses_e), np.array(ses_m),
105         float(np.polyfit(np.log(h[tail]), np.log(np.
106             maximum(np.array(errors_e)[tail], np.finfo(
107                 float).tiny)), 1)[0]),
108         float(np.polyfit(np.log(h[tail]), np.log(np.
109             maximum(np.array(errors_m)[tail], np.finfo(
110                 float).tiny)), 1)[0]))

```